CSC490 Final Report - Group 8

Isaac Abell - 1008831329, Thomas (Yehyun) Lee - 1008992217,
Zeling (Zoey) Zhang - 1007714779, Jessica Zhang- 1008494130

Table of Contents

1. Abstract 1
2. Introduction 1
3. Problem statement 1
4. Related work 2
4.1 Dance Learning and Performance Feedback 2
4.2 Computational Dance Understanding: Motion, Music, and Segmentation 2
4.3 Datasets and Systems for Dance Analysis 3
5. Technical approach, solution, and designs 3
5.1 Proposed Solution 3
5.2 Algorithm Pipeline 4
5.2.1 Temporal Synchronization via Audio Alignment 4
5.2.2 Structural Segmentation via Downbeat Estimation 4
5.2.3 Body Segmentation and Normalization 5
5.2.4 Feedback 5
5.3 Architecture 5
5.3.1 Architecture Overview 5
5.3.2 Architectural Considerations 6
6. Experiments, key metrics, and results 6
6.1 Choosing a proper audio alignment algorithm 6
6.1.1 Performance Comparison 6
6.1.2 Final Selection 7
6.2 Baseline Evaluation of Multimodal Feedback Models 7
6.2.1 Experimental Setup 7
6.2.2 Evaluation Metrics Design 8
6.2.3 Results 8
6.2.4 Discussion and Forward Direction 9
8. Team reflection on the project including connection to the course material 10
Appendix A: User Study Report Summary 11
Appendix B: Detailed Results for Multimodal Baseline Models 12
References 13

1. Abstract

As AI-aided apps bloom, we notice a gap in the field of dancing. Our work thus attempts to bridge this gap by developing an AI-aided tool that captures subtle differences that are difficult to spot by dancers to address the lack of feedback resources available to dancers. By conducting a landscape analysis, we examined existing work on performance feedback, found technologies to support computational dance understanding in motion, music, and segmentation, and composed datasets and systems for dance analysis. Following, we propose a framework that uses audio alignment to auto-align the user video with the reference video and leverages off-the-shelf technologies like YOLO to generate overlays. We developed a normalizing module to align the overlays and pass the processed data into feedback pipelines to generate easy-to-understand feedback. Our experimental results supported the choice of the Smith-Waterman algorithm with start/end padding for audio alignment and the use of the gemini 2.5 flash lite for multimodal feedback. For future directions, we discussed potential improvements, such as expanding the dimension of feedback. Lastly, as a wrap-up for the course, our team reflects on how this process relates to the course material.

2. Introduction

Many dancers can reproduce the choreography but still feel that their performance looks “off” compared to the reference. The missing difference is often not the steps themselves, but style-level execution: timing, posture, weight transfer, dynamics, and movement quality. These mismatches are especially hard to diagnose in self-practice, where dancers can see that something is different but cannot easily identify the cause.

Existing practice tools only partly address this problem. Mirrors provide immediate visual feedback but not temporal inspection. Manual video review supports replay, but it is slow, subjective, and mentally demanding. Coaching can help, but it is not always accessible, and even experienced dancers may struggle to clearly articulate the exact source of a mismatch. As a result, many dancers are left to repeatedly imitate the reference without knowing what to correct.

This project addresses that gap by developing a dance analysis pipeline that compares a user’s performance video with a reference choreography clip and identifies fine-grained differences in execution at the level of learnable movement segments. Rather than treating dance comparison as a generic pose-matching task, the system is designed around choreography practice: alignment to music, segmentation into beat-level motion units, and localized timing feedback. The goal is to move beyond coarse similarity scores toward feedback that can support real correction during practice.

3. Problem statement

Current dance practice workflows are poorly suited to identifying fine-grained performance errors. A dancer may reproduce the overall choreography yet still look noticeably different from the reference because of subtle mismatches in timing, posture, control, coordination, or movement quality. These differences are often visible in performance but difficult to diagnose through mirrors, manual video review, or informal coaching. The challenge is therefore not just to compare two dance videos, but to do so in a way that is temporally aligned, robust to recording differences, and sensitive to the execution details that matter in dance. In this project, we address that challenge by building a pipeline that aligns a

user video with a reference, segments the choreography into analyzable movement units, and detects localized mismatches that can support actionable feedback during practice.

4. Related work

4.1 Dance Learning and Performance Feedback

Prior work on dance learning technologies shows that computational systems can support practice by comparing a learner's movement against a reference, but most emphasize overall scoring, synchronization, or broad similarity rather than localized corrective feedback (Alexiadis et al., 2011; Anderson et al., 2013; Zhou et al., 2021). Early systems using Kinect-based skeleton tracking demonstrated that dance performance could be evaluated computationally, though the output was primarily evaluative rather than diagnostic (Alexiadis et al., 2011). In HCI, systems such as YouMove showed that augmented replay and guided review can improve movement learning, but they focused more on instructional support than on identifying the specific local mismatches that make a performance visibly off (Anderson et al., 2013).

More recent work moves closer to feedback-oriented dance assessment. SyncUp highlights problematic body regions and temporal intervals, while DanceMVP and later multimodal transformer-based approaches extend this direction toward richer performance assessment and progression tracking (Zhou et al., 2021; Zhong & Demiris, 2024; Chen, 2025). However, these systems still primarily assess performance rather than helping learners decide what to correct next. Prior qualitative work on online dance learning with teacher-AI cooperation further suggests that AI feedback is only useful when it is interpretable within real learning workflows (Kang et al., 2023). Our work builds on this literature but shifts the focus from global evaluation to segment-level, actionable feedback for dance learning.

4.2 Computational Dance Understanding: Motion, Music, and Segmentation

The technical foundation of our system draws from pose-based motion analysis, rhythm-aware alignment, and choreography segmentation. Pose estimation offers a practical representation for comparing body movement across videos, with OpenPose and MediaPipe providing strong baselines for extracting human keypoints in both offline and real-time settings (Cao et al., 2017; Lugaresi et al., 2019). Once dance is represented as pose trajectories, temporal structure becomes central, since dance is defined not only by spatial configuration but also by rhythmic timing.

This makes beat tracking and segmentation especially important for learning-oriented systems. Prior work in beat and downbeat tracking shows that metrical structure can be estimated reliably enough to support rhythm-aware analysis (Böck et al., 2016). Recent multimodal music modeling similarly supports treating rhythm as an explicit signal rather than an incidental feature (Liu et al., 2025). In parallel, automatic dance video segmentation work argues for dividing choreography into smaller, meaningful units using audio and visual cues, which aligns closely with our decision to treat beat-to-beat transitions as the unit of comparison and feedback rather than relying on whole-clip similarity or choreography generation objectives (Endo et al., 2024; Li et al., 2021).

4.3 Datasets and Systems for Dance Analysis

Existing datasets have driven major progress in dance understanding, but most were created for choreography generation, genre recognition, or general motion analysis rather than learner-facing feedback. AIST Dance Video Database, AIST++, and FineDance are valuable for representation learning, while ImperialDance and more recent assessment- and segmentation-oriented work move closer to evaluation and structure (Tsuchida et al., 2019; Li et al., 2021; Li et al., 2023; Zhong & Demiris, 2024; Chen, 2025; Endo et al., 2024). Industry tools reveal a similar gap: platforms such as STEEZY Studio provide useful practice affordances like looping, playback control, and view switching, and newer AI-assisted systems such as Dalea and SyncTrainer suggest demand for automated comparison, yet public systems still largely emphasize self-review, synchronization, or scoring over fine-grained, body-part-specific corrective feedback (STEEZY Studio, 2022; Dalea, n.d.; Google Play, 2023; Ubisoft, 2022). This gap motivates our approach: a segment-level feedback pipeline designed not just to assess dance performance, but to help users improve it.

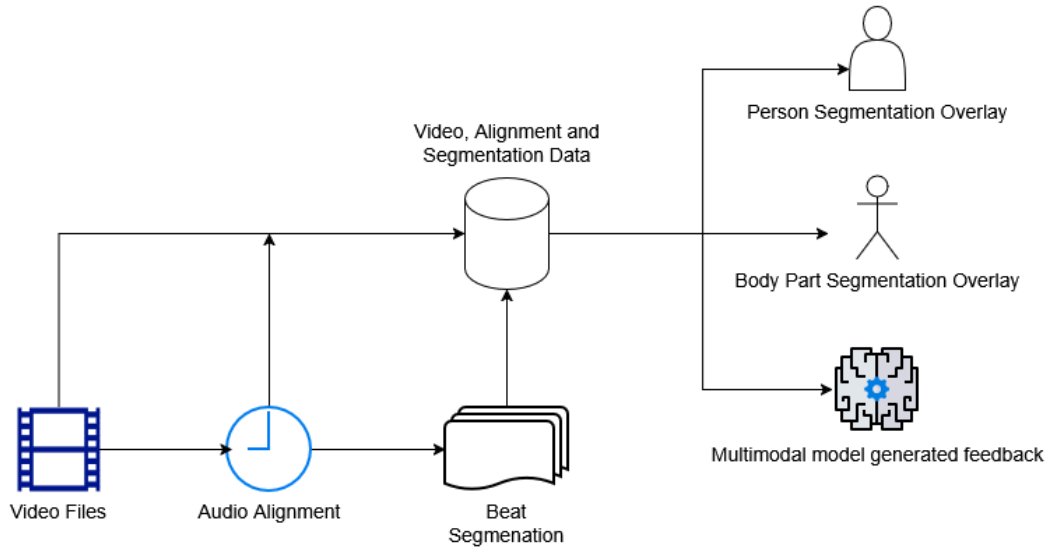
5. Technical approach, solution, and designs

5.1 Proposed Solution

Our proposed solution is a dance learning web application that lets users compare a practice video against a reference choreography and receive structured feedback on performance mismatches. It is designed around a key finding from our user study (n=9n=9n=9; see Appendix A for full details): dancers want automated feedback, but their current workflow still depends heavily on manual self-correction. In our study, **75%** of participants relied mainly on mirrors or self-monitoring, and many cited difficulty identifying their own mistakes as a central challenge. In response, we designed the product not as a simple scoring system, but as a practice-oriented tool that supports iterative review, targeted correction, and repeated rehearsal.

The interface was built to match realistic dance practice settings. Although **88.9%** of participants currently record on their phones, we chose a **web app** to support both lightweight capture and larger-screen review, especially since **44.4%** reported practicing in a studio. The workflow guides users from **video upload** to **kinematic analysis** to a **review panel** with speed control, looping, and thumbnail-based navigation. Because **66.7%** of participants commonly record from a low angle, we also included **overlay and split-screen comparison modes with beat-aligned playback** to make mismatches easier to inspect despite perspective differences. To address concerns that automated feedback might be too generic, we designed a **two-layer feedback system**: a high-level summary for quick correction and detailed coaching cards for granular diagnosis, aligned with the **87.5%** of users who preferred micro-level feedback. We also added **Beginner, Standard, and Advanced modes** so the technical depth of the feedback can better match the user's experience level.

5.2 Algorithm Pipeline



5.2.1 Temporal Synchronization via Audio Alignment

For the audio alignment algorithm implementation, a custom diagonal-restricted local alignment algorithm (a gapless modification of the Smith-Waterman algorithm) was used. Standard sequence alignment algorithms permit horizontal and vertical matrix traversals to account for insertions and deletions. However, because the underlying tempo and structure of the reference music remain constant across both videos, time-warping or gap penalties are unnecessary; the user's audio is simply offset by a fixed temporal lag.

By constraining the matrix traversal strictly to the diagonal, the algorithm acts as a highly optimized gapless alignment, perfectly mapping to the linear time-shift of the audio tracks while significantly reducing computational complexity compared to the original Smith-Waterman algorithm. In our testing sets, this constrained algorithm successfully aligned 99.7% of the samples within 1/60 of a second from the true alignment.

Because this method still relies on a cumulative local scoring matrix, it naturally tends to truncate the actual boundaries of the sequence (starting slightly late and ending early as the correlation score ramps up or tapers off). To counteract this, a padding buffer is appended to the predicted start and end timestamps, ensuring complete capture of the alignment window.

5.2.2 Structural Segmentation via Downbeat Estimation

The segmentation pipeline relies on acoustic analysis to identify musical phrasing. We utilize librosa.beat.beat_track utility to estimate the global tempo and locate beat events. To isolate the 8-count phrases, the system must identify the "downbeat" (Count 1).

The algorithm operates on the heuristic that the downbeat within an 8-count musical phrase typically exhibits the highest acoustic intensity. It evaluates all eight potential beat-index phase offsets (1 through 8) and calculates the average onset strength for each. The phase offset yielding the maximum average

onset strength is designated as the downbeat. These identified downbeats are then translated into specific timestamps to structurally segment the video files.

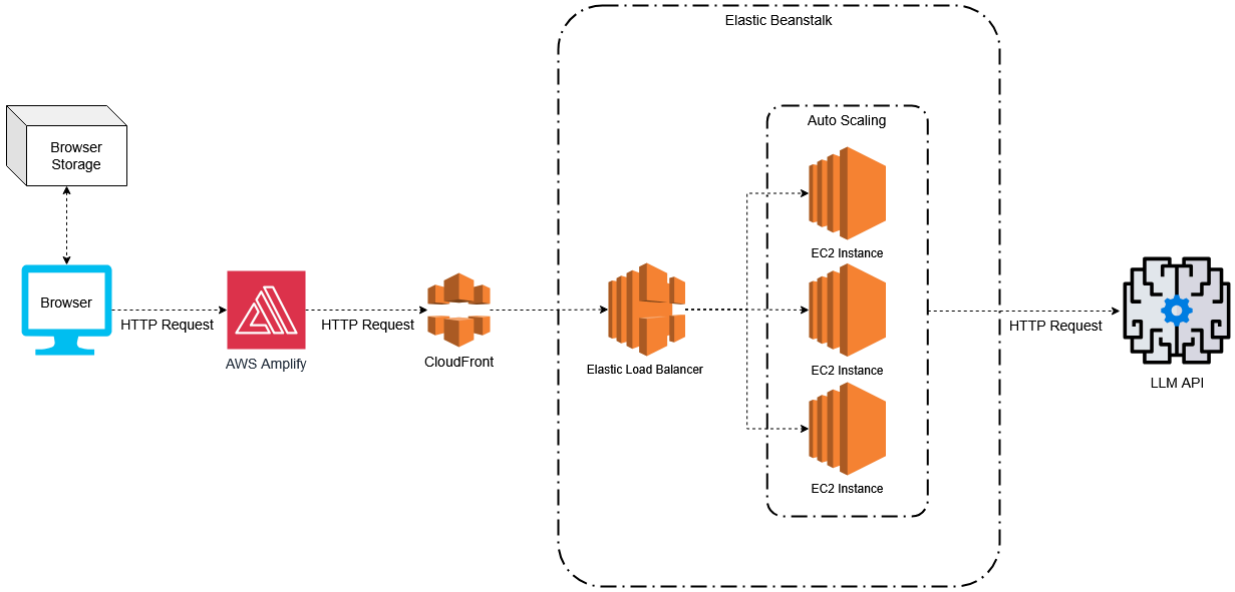
5.2.3 Body Segmentation and Normalization

For body segmentation, we ultimately chose **YOLO** because it offered the best balance between **speed, segmentation quality, and overall user experience**. Compared with **SAM3**, YOLO was substantially faster and therefore better suited for an interactive feedback pipeline where latency directly affects usability. Compared with **BodyPix**, YOLO delivered much stronger segmentation quality, especially in challenging cases such as **fast choreography** and **multi-dancer frames**, where BodyPix often produced highly inaccurate or unstable masks despite its speed advantage. In practice, this made YOLO the most suitable option: it was fast enough for a responsive system while still providing segmentation accuracy robust enough to support reliable downstream pose and feedback generation.

5.2.4 Feedback

For the feedback stage, the final solution uses a **per-segment, kinematically grounded input pipeline** rather than relying solely on raw video. For each segment, we pass the model a structured package comprising the **system prompt**, **YOLO-based segmentation output**, **YOLO pose estimates**, and derived **kinematic features**, along with supporting metadata such as **normalization parameters, body scale, translation offsets, and keypoint confidence scores**. This design provides the model with explicit information about body position, movement, and spatial alignment across the reference and user clips, enabling feedback to be generated from structured motion evidence rather than from unconstrained pixel-level inference.

5.3 Architecture



5.3.1 Architecture Overview

Component Flow

- **Frontend & State Management:** The frontend application is hosted on AWS Amplify. For the MVP, user state and data are managed via local Browser Storage to provide a seamless local experience.
- **Backend Delivery & Routing:** Backend API requests from the browser are routed through CloudFront, which forwards them to an Elastic Load Balancer (ELB) managed within an AWS Elastic Beanstalk environment.
- **Compute & Scaling:** The ELB distributes incoming traffic to EC2 instances. Multiple instances are deployed strictly for Auto Scaling purposes to handle fluctuating traffic.
- **AI Integration:** To power the application's AI features, the EC2 instances make outbound HTTP requests to an external LLM API.

Design Rationale

- **Local-First MVP:** Relying on Browser Storage eliminates the immediate need for a persistent database, accelerating the MVP timeline.
- **Backend Edge Optimization:** Positioning CloudFront in front of the backend application ensures optimized network routing and secure, efficient delivery of API requests.
- **Elastic Scalability:** The combination of Elastic Beanstalk and Auto Scaling ensures the backend adapts to traffic spikes without over-provisioning resources.
- **Cost-Effective SOTA AI:** By calling an external LLM API, we allow the application to leverage state-of-the-art models without incurring the costs or complexity of hosting these large models internally.

5.3.2 Architectural Considerations

While the primary application ecosystem is built in TypeScript, the audio alignment and beat segmentation modules presented a computational bottleneck. Both algorithms rely heavily on dynamic programming and large matrix arithmetic. To optimize performance, these specific modules were implemented as a separate service in Python. NumPy executes these matrix operations exponentially faster than TypeScript's runtime, yielding a significant reduction in computation time which outweighs the added complexity of managing 2 separate backend codebases. In the final prototype, we ran the backend of the YOLO overlay using a single Elastic Beanstalk instance, which was an engineering tradeoff because the asynchronous overlay jobs use in-memory state, so correctness and stability are guaranteed if we run the jobs on a single instance, avoiding the need to add extra infrastructure such as a shared job store. This is very easy to change the config, so we've added a note in the future work section.

6. Experiments, key metrics, and results

6.1 Choosing a proper audio alignment algorithm

To achieve precise audio clip alignment, we evaluated two candidate algorithms: an Onset-Envelope Cross-Correlation (OECC) approach and a modified Smith-Waterman (MSW) algorithm. The selection process prioritized alignment accuracy, which is essential for the system's core functionality, while balancing computational complexity.

6.1.1 Performance Comparison

- **Computational Complexity:** The OECC algorithm proved to be computationally lighter, operating at an O($Nlog(N)$) time complexity (where N is the length of the audio). In contrast, the MSW algorithm is more resource-intensive, with a time complexity of O(N^2).

- **Alignment Accuracy:** Despite its slower execution time, the MSW algorithm demonstrated superior accuracy. Against our testing set, the MSW algorithm aligned clips within 1/60 of a second 99.7% of the time, establishing a highly reliable baseline. The OECC algorithm only achieved accuracy within this threshold 93.6% of the time and was prone to catastrophic alignment failures on specific test cases.

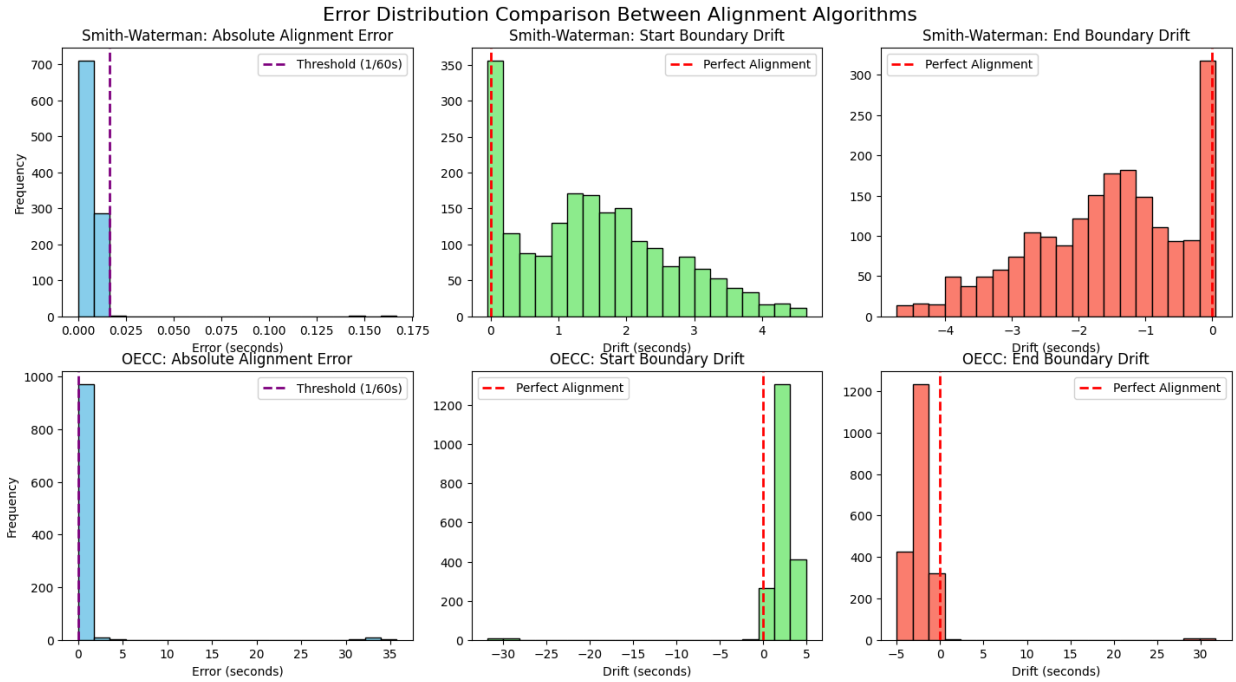


Figure 1: Error distributions between audio alignment algorithms

As illustrated in Figure 1, the error distributions highlight the difference in reliability. The MSW algorithm maintains a tight alignment error distribution (Top Left), whereas the OECC algorithm exhibits more variance and significant outliers (Bottom Left).

6.1.2 Final Selection

Given that precise audio alignment is a strict requirement for this system, the accuracy gains of the modified Smith-Waterman algorithm outweigh its higher computational cost. Therefore, the MSW algorithm with start/end padding was selected as the production solution.

6.2 Baseline Evaluation of Multimodal Feedback Models

6.2.1 Experimental Setup

We conducted a baseline evaluation of four multimodal models, **Gemini 2.5 Flash-Lite, Gemini 2.5 Flash, Gemini 2.5 Pro, and GPT-5.2**, to assess their suitability for automated dance micro-timing feedback under a shared input protocol. All models received the same system prompt to compare a reference video with a user video at the **move-window** level, where each move was defined as the transition between two consecutive beats. For each window, models were asked to assess **initiation, peak**

motion, and landing, explicitly reference left/right body parts and weight state, and return a single timing label from a fixed vocabulary in structured JSON format.

Inputs consisted of **per-segment video clips** with corresponding move-window timestamps. The test video was intentionally controlled: high-resolution, stable single-camera view, single dancer, and minimal visual noise, to ensure that the observed limitations reflected model behavior rather than degraded input quality. For each model call, we stored the raw JSON output and associated metadata for later analysis. Outputs were anonymized as **Model A–D** during review to reduce anchoring bias. **Seven intermediate dancers** served as raters, selected to reflect the intended user group rather than expert coaches, since the latter would likely apply different standards of timing sensitivity and language tolerance.

6.2.2 Evaluation Metrics Design

Four rubric dimensions were rated for each move-model pair on a **1–5 integer scale**:

- **Label accuracy**: whether the categorical timing label (for example, *early*, *late*, or *rushed*) correctly describes the observed micro-timing deviation.
- **Body part specificity**: whether the feedback correctly identifies the relevant limbs, laterality (left/right), and weight state involved in the mistimed transition.
- **Coaching actionability**: whether the coaching_note provides information a dancer could use to make a correction, rather than merely restating the error.
- **Confidence calibration**: whether the model's stated confidence level appropriately matches the ambiguity of the visual evidence.

These dimensions were chosen to capture both **diagnostic performance** (label accuracy and body part specificity) and **pedagogical usefulness** (actionability and confidence calibration). This distinction is important because technically correct feedback still has limited value if it is vague, overly technical, or difficult for a learner to act on.

6.2.3 Results

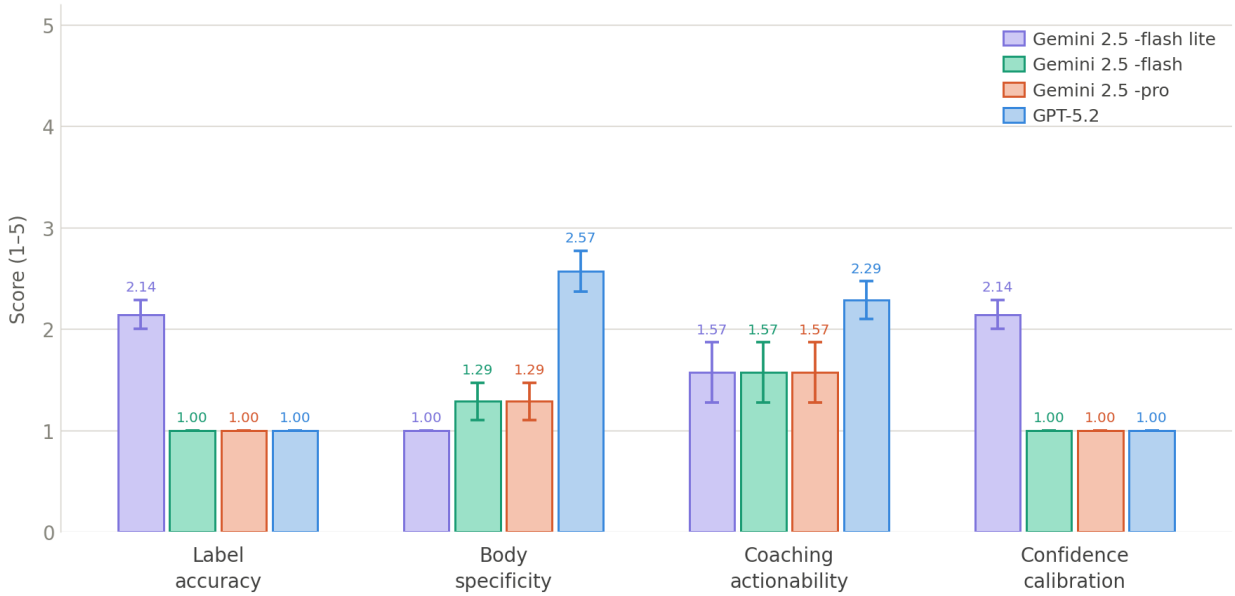


Figure 2. All metrics, mean ± SEM (n = 7 per cell)

Figure 2 shows the mean ± SEM for all four metrics across all four models (n = 7 ratings per model per metric). Full per-metric analyses and example outputs are provided in the Appendix. **All models performed poorly under the current input protocol**, with scores in the **1–2 range** across all metrics on a 1–5 scale. For **label accuracy**, Gemini 2.5 Flash-Lite scored highest (**2.14 ± 0.14**), while Gemini 2.5 Flash, Gemini 2.5 Pro, and GPT-5.2 were all at floor. For **body part specificity**, GPT-5.2 scored highest (**2.57 ± 0.20**), compared with **1.00–1.29** for the Gemini models. For **coaching actionability**, all models remained low (**1.57–2.29**), with GPT-5.2 again highest (**2.29 ± 0.18**). For **confidence calibration**, Flash-Lite scored highest (**2.14 ± 0.14**), while the remaining models were again at low.

Qualitatively, the models failed in different ways. The **Gemini family** produced largely template-like, low-specificity feedback that was structurally consistent but weak in diagnostic value. **GPT-5.2** generated more anatomically specific outputs, often naming laterality and precise landmarks, but its feedback was frequently too technical for the target user group. Thus, higher specificity did not translate into proportionally higher actionability.

6.2.4 Discussion and Forward Direction

The baseline finding is that **no tested model achieves operationally useful performance under the current input protocol**. This is supported by both the quantitative results and the qualitative error patterns: all models scored in the 1–2 range, label accuracy was at or near floor for three of the four models, and the generated feedback was either too generic (Gemini family) or too technical (GPT-5.2).

We attribute the primary bottleneck to the **lack of structured pose information in the input**. The models currently receive raw video clips and timestamp windows, but no explicit kinematic grounding such as joint positions, body scale, or translation. This baseline led to two decisions for our final product. First,

for **accuracy**, we will test **skeletal and kinematic augmentation within the Gemini family**, adding pose metadata derived from a **YOLO-based** segmentation and pose estimation pipeline. We retained Gemini for this phase on cost grounds, since Flash-Lite, Flash, and Pro showed no meaningful accuracy separation under the current protocol. Second, for **feedback quality**, we will test **prompt variation and language simplification**, especially prompts that enforce colloquial studio language and actionable coaching style. Because GPT-5.2 showed the strongest body-part specificity, it remains the most promising candidate for a later prompt-focused follow-up once the representation bottleneck has been tested.

7. Future work and next steps

1. Incorporate more feedback dimensions

While the current iteration prioritizes micro-timing, our user study identified four additional critical dimensions, including grounding and isolation, that are essential for professional-level performance. Future work will involve training the diagnostic engine to incorporate these qualitative metrics, providing a holistic evaluation of a dancer's performance beyond simple temporal accuracy.

2 Incorporate more camera angles

Our current work is trained and tested on a single front facing angle only, however due to the variety of filming conditions identified in our survey (specifically low-angle mobile recordings), we plan to implement robust 3D pose normalization. By transitioning from 2D pose estimation to depth-aware mapping, the system will better accommodate non-standard camera angles and provide more accurate feedback regardless of the user's hardware setup.

3. Provide live-time feedback

We aim to transition from asynchronous analysis to real-time coaching by leveraging low-latency models like Gemini Live. To support this, the infrastructure will shift from local-only storage to a persistent cloud-based architecture. This will enable cross-device synchronization and long-term progress tracking without exhausting local device storage.

4. Saving dances in the cloud

Currently, our implementation only allows users to save data locally, however we would like to add persistent cloud storage to enable users to view their past dances across multiple devices and to prevent them from taking up significant local storage with past dance sessions.

5. Multidancer mode

Future updates can expand to improving group environments by implementing person-reidentification (ReID) and multi-object tracking. This will allow the app to isolate and analyze a specific dancer within a group or troupe, providing personalized feedback even in crowded studio settings.

6. Scale

We plan to replace the current in-memory overlay job management with shared cloud storage and coordination so that the YOLO backend can scale cleanly across multiple Elastic Beanstalk instances.

8. Team reflection on the project including connection to the course material

Our biggest takeaway from this project was that building an ML product for dance learning is fundamentally a **systems design problem**, not a pure model selection problem. Early on, it was tempting

to think that the main challenge was choosing the strongest vision model or the best LLM for feedback. The course material pushed us toward a broader view: an ML system has to be reasoned about across data, infrastructure, model, application, UX, use case, domain knowledge, and hardware rather than as an isolated model artifact (Linkov, 2026a; Linkov, 2026g).

That framing matched our project closely, because the quality of the final feedback depended just as much on segmentation, alignment, data representation, and latency as on the model itself. One concrete way this changed our thinking was in how we treated intermediate representations. In the final pipeline, we did not rely only on raw video; we used segmented clips, pose estimates, normalized motion representations, and structured body-position information as intermediate layers before feedback generation. This closely relates to the course discussion of feature stores and reusable ML data layers that make downstream systems more reliable (Linkov, 2026d). Although we did not build a literal feature store, we arrived at the same systems insight: downstream feedback quality depended on having a disciplined, structured representation layer rather than passing unprocessed inputs directly into a model.

Over the course of the project, one of our biggest lessons was that **problem definition and scope control are themselves core technical tasks in ML product development**. At the beginning, we were interested in a much broader set of features, including basic post-hoc feedback, live feedback, and even downstream ideas such as recommending dance masters based on the kinds of mistakes a user makes. As the course progressed, however, we became much more aware that each of these features was not a single problem but a stack of technically difficult subproblems involving representation, latency, evaluation, and product usefulness. The course material helped us see that strong ML systems are not built by pursuing every technically interesting direction at once, but by prioritizing the capabilities that best match the user problem and the constraints of the system (Linkov, 2026a; Linkov, 2026d; Linkov, 2026g).

That perspective shaped our process directly. We spent a significant amount of time reviewing literature, identifying multiple feedback dimensions that could plausibly help dancers, and then using user study results to decide which dimension was most valuable and feasible within the project timeline. Combined with our decision to analyze choreography through localized movement units, this scoping process led us toward a narrower but more coherent problem formulation. It also helped us notice that our approach works especially well for certain dance genres, which, in hindsight, suggests that not committing too early to a single genre was beneficial. More broadly, this project taught us that ML engineering is not only about building models, but about learning how to narrow an ambitious idea into a technically defensible, evaluable, and genuinely useful system. We also found that this discovery process was one of the most valuable parts of the project and something we would want to continue beyond the course.

Appendix A: User Study Report Summary

Link to results: User Study

- Category Result: Intermediate (62.5%)
- Top 3 Styles: J-pop, jazz, kpop
- Frequency: Quartered (no daily)
- Consistent Training (Top 2): <1 hour, 1–2 hours (followed by 10+)
- Usage: 88.9% for learning choreography
- Angle: Mostly full body (66.7% low angle due to lack of stand; 44.4% eye level)

- Device: Phone (88.9%)
- Location: Home (55.6%) and Studio (44.4%)
- Importance for High-Quality Performance:
    - Isolation: Overwhelmingly 5
    - Grounding & Attack: Varied
    - Micro-timing & Groove Phase: Lower end (1 and 2 respectively)
- Importance for Distinguishing Advanced/Pro-Level Dancing:
    - 5: Attack
    - 4: Groove
    - 3–5 (Same Level): Micro-timing and isolation
    - Grounding: Still varied
- Most Commonly Overlooked by Learners: Attack (55.6%)
- Context for Technique Importance: Stage/teaching students (62.5%)
- App to Focus on First: Timing, attack
- Biggest Pain Points: Hard to notice micro-mistakes on my own, no clean drills for improvement, camera/recording issues
- Hardest to Judge: Energy / performance quality
- Desired Features (Score of 5): Privacy mode, beat-aligned comparison
- Feedback Format: Both summary + details (50%), followed by 1–3 top issues per phrase/coaching cards (37.5%)
- App Adaptation Goals: Prioritize musicality and groove, followed by cleanliness/sharpness

Appendix B: Detailed Results for Multimodal Baseline Models

Table 1. Quantitative Result for multimodal feedback models

Metric	Result
Label accuracy	Gemini 2.5 Flash-Lite achieved the highest mean label accuracy (**2.14 ± 0.14**), while Gemini 2.5 Flash, Gemini 2.5 Pro, and GPT-5.2 all remained at floor (**1.00 ± 0.00**). This does not suggest stronger perception from Flash-Lite, but rather more conservative labeling: its broader, lower-confidence outputs were scored as less incorrect than the larger models' confident but systematically wrong labels. In effect, Flash-Lite benefited from a hedging strategy that reduced catastrophic label errors. Even so, all models remained in the **1–2 range**, well below practical usefulness.
Body part specificity	GPT-5.2 was the clear outlier here, scoring **2.57 ± 0.20**, compared with the Gemini models' **1.00–1.29**. This matches qualitative observations: GPT-5.2 more often identified specific anatomical landmarks and sides, while Gemini outputs stayed at the level of broader body regions. GPT-5.2 also showed higher variance across move windows, suggesting that this specificity was applied inconsistently.
Coaching actionability	All models scored similarly in the **1.57–2.29** range. GPT-5.2 again ranked highest (**2.29 ± 0.18**), but with a smaller advantage than in body specificity. This indicates that greater anatomical precision did not translate proportionally into more actionable coaching.
Confidence	Gemini 2.5 Flash-Lite again scored highest (**2.14 ± 0.14**), while the other models

calibration	stayed at floor. This is consistent with its more cautious output style: it was more likely to signal low confidence on ambiguous move windows, especially near segment boundaries.

Table 2. Qualitative Result for multimodal feedback models

Model	Result
Gemini Family	**Template-like, low-specificity outputs.** Across all three Gemini variants, feedback was structurally similar across move windows and segments. Phrases such as "the transition begins slightly ahead of the beat" and "weight transfer is initiated early" repeated with only minor variation, often regardless of the clip's specific biomechanics. While generally correct at a broad category level, these responses offered weak diagnostic value, making it hard for dancers to identify exactly what to fix. This likely reflects instruction-following behavior under a constrained JSON schema, which encouraged consistency but reduced clause-level specificity.
GPT-5.2	**Domain-specific vocabulary with an accessibility deficit.** GPT-5.2 produced more anatomically and temporally specific feedback, with stronger use of laterality and sub-event detail. However, raters consistently found its coaching language too technical for the target users. Terms and phrasing often resembled academic or clinical descriptions rather than studio instruction, which reduced actionability even when the body references were accurate. Intermediate dancers, in particular, rated these responses as less useful for correction than for identification. A possible confound is that GPT-5.2 received video through frame extraction rather than native video input, which may have reduced access to temporal cues and contributed to variation across move windows.

References

Alexiadis, D. S., Kelly, P., Daras, P., O'Connor, N. E., Boubekeur, T., & Ben Moussa, M. (2011). *Evaluating a dancer's performance using Kinect-based skeleton tracking*. In Proceedings of the 19th ACM International Conference on Multimedia.

Anderson, F., Grossman, T., Matejka, J., & Fitzmaurice, G. (2013). *YouMove: Enhancing movement training with an augmented reality mirror*. In Proceedings of the 26th Annual ACM Symposium on User Interface Software and Technology.

Böck, S., Krebs, F., & Widmer, G. (2016). *Joint beat and downbeat tracking with recurrent neural networks*. In Proceedings of the International Society for Music Information Retrieval Conference.

Cao, Z., Simon, T., Wei, S.-E., & Sheikh, Y. (2017). *Realtime multi-person 2D pose estimation using part affinity fields*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition.

Chen, L. (2025). *Visual language transformer framework for multimodal dance performance evaluation and progression monitoring*. Scientific Reports, 15, Article 30649.

Dalea. (n.d.). *AI-powered dance practice at home*.

Endo, K., Tsuchida, S., Fukusato, T., & Igarashi, T. (2024). *Automatic dance video segmentation for understanding choreography*. arXiv.

Google Play. (2023). *SyncTrainer* [App listing].

Kang, J., Kang, C., Yoon, J., Ji, H., Li, T., Moon, H., Ko, M., & Han, J. (2023). *Dancing on the inside: A qualitative study on online dance learning with teacher-AI cooperation*. Education and Information Technologies, 28, 12111–12141.

Li, R., Yang, S., Ross, D. A., & Kanazawa, A. (2021). *AI Choreographer: Music conditioned 3D dance generation with AIST++*. In Proceedings of the IEEE/CVF International Conference on Computer Vision.

Li, R., Zhao, J., Zhang, Y., Su, M., Ren, Z., Zhang, H., Tang, Y., & Li, X. (2023). *FineDance: A fine-grained choreography dataset for 3D full body dance generation*. In Proceedings of the IEEE/CVF International Conference on Computer Vision.

Linkov, D. (2026a, January 12). *Lecture 2: Cloud infrastructure* [PowerPoint slides]. CSC490 ML Engineering Capstone, University of Toronto.

Linkov, D. (2026b, January 19). *Lecture 3: Build vs. buy* [PowerPoint slides]. CSC490 ML Engineering Capstone, University of Toronto.

Linkov, D. (2026c, January 26). *Lecture 4: Prompting and constrained decoding* [PowerPoint slides]. CSC490 ML Engineering Capstone, University of Toronto.

Linkov, D. (2026d, February 9). *Lecture 6: Feature store and evaluation metrics* [PowerPoint slides]. CSC490 ML Engineering Capstone, University of Toronto.

Linkov, D. (2026e, February 23). *Lecture 7: Model training* [PowerPoint slides]. CSC490 ML Engineering Capstone, University of Toronto.

Linkov, D. (2026f, March 2). *Lecture 8: Reinforcement learning for LLMs* [PowerPoint slides]. CSC490 ML Engineering Capstone, University of Toronto.

Linkov, D. (2026g, March 9). *Lecture 9: Data Eng and ML Systems* [PowerPoint slides]. CSC490 ML Engineering Capstone, University of Toronto.

Liu, S., Ren, L., & Zhao, J. (2025). *Joint modeling of rhythm and emotion in music via frequency-domain attention and multi-task learning*. Multimedia Systems, 32, Article 27.

Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., … Grundmann, M. (2019). *MediaPipe: A framework for building perception pipelines*. arXiv.

STEEZY Studio. (2022). *Taking a class* [Help center article].

Tsuchida, S., Fukayama, S., Hamasaki, M., & Goto, M. (2019). *AIST Dance Video Database: Multi-genre, multi-dancer, and multi-camera database for dance information processing*. In Proceedings of the International Society for Music Information Retrieval Conference.

Ubisoft. (2022). *Revolutionising motion gaming: The camera controller feature*.

Zhong, Y., & Demiris, Y. (2024). *DanceMVP: Self-supervised learning for multi-task primitive-based dance performance assessment via transformer text prompting*. Proceedings of the AAAI Conference on Artificial Intelligence, 38(9), 10270–10278.

Zhou, Z., Xu, A., & Yatani, K. (2021). *SyncUp: Vision-based practice support for synchronized dancing*. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 5(3), Article 143.